

Empowered AI

Project Team

Hamza Tariq 21I-0707
Abdul Wasay 21I-0834
Hassan Abbas 21I-0507

Session 16/5/2025

Supervised by

Dr. Usman Haider

Co-Supervised by

Ms. Sidra Fayyaz



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Existing Solutions	1
1.2	Problem Statement	3
1.3	Motivation	3
1.4	Problem Solution	3
1.5	Scope	5
1.6	Modules	5
1.7	Work Division	7
2	Project Requirements	8
2.1	Event Response Table	8
2.2	Functional Requirements	9
2.2.1	Module 1: Vision Insights	9
2.2.2	Module 2: Smart Interface	9
2.2.3	Module 3: Live Assist	9
2.2.4	Module 4: Emotion Sense	9
2.2.5	Module 5: Smart Assist Enhancements	10
2.3	Non-Functional Requirements	10
2.3.1	Reliability	10
2.3.2	Usability	10
2.3.3	Performance	11
2.3.4	Security	11
3	System Overview	12
3.1	Architectural Design:	12
3.1.1	Explanation of Architecture Diagram:	12
3.2	Data Design	13
3.2.1	Mobile Device	14
3.2.2	Image Transmission	14
3.2.3	Server Processing and Storage	14
3.2.4	Results Transmission	14
3.2.5	User Data Storage Feedback	15
3.2.6	Overall Workflow	15

3.2.7	Database Selection	15
3.3	Domain Model	15
3.3.1	Entities and Their Attributes	16
3.3.2	Relationships	18
3.3.3	Cardinalities	18
3.4	Design Models	19
3.4.1	Activity Diagram	19
3.4.1.1	Activity Diagram 1: Object Detection	19
3.4.1.2	Activity Diagram 2: Scene Description	20
3.4.1.3	Activity Diagram 3: Text Recognition	21
3.4.1.4	Activity Diagram 4: Emotion Detection	22
3.4.1.5	Activity Diagram 5: Currency Detection	23
3.4.1.6	Activity Diagram 6: Item Locator	24
3.4.1.7	Activity Diagram 7: Color Detection	26
3.4.2	Class Diagram	26
3.4.3	Class-level Sequence Diagram	30
3.4.4	State Transition Diagram	32
4	Implementation and Testing	34
4.1	Algorithm Design	34
4.2	External APIs/SDKs	36
4.3	Testing Details	37
4.3.1	Unit Testing	37
	References	44

List of Figures

3.1	Architecture Diagram	13
3.2	Data Design Diagram	13
3.3	Domain Model	16
3.4	Object Detection Activity Diagram	20
3.5	Scene Description Activity Diagram	21
3.6	Text Recognition Activity Diagram	22
3.7	Emotion Detection Activity Diagram	23
3.8	Currency Detection Activity Diagram	24
3.9	Currency Detection Activity Diagram	25
3.10	Color Detection Activity Diagram	26
3.11	Class Diagram	27
3.12	Class-Level Sequence Diagram	31
3.13	State Transition Diagram	32
4.1	Object Detection Algorithm	34
4.2	Scene Description Algorithm	35
4.3	Text Recognition Algorithm	35
4.4	Facial Emotion Algorithm	36
4.5	Audio Response Algorithm	36

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Team Member Responsibilities	7
2.1	User Interaction Events with System Responses	8
4.1	List of External APIs and SDKs used in EmpoweredAI	37
4.2	Test Case for User Registration	38
4.3	Test Case for Duplicate Email Registration	39
4.4	Test Case: User Login	40
4.5	Test Case: Invalid Login Attempt	41
4.6	Test Case: Scene Description from Image	42

Chapter 1

Introduction

“**Empowered-AI**” project is a complex accessibility application that responds to the needs of disabled persons in an effort to improve their independence and the quality of life of the visually impaired. The app incorporates recent AI technology to facilitate communication help and interaction support, interaction helps throughout a vast number of everyday activities. With features such as computer vision, natural language processing, and haptic feedback, the “Empowered-AI” seeks to remove communication restrictions and make daily life less challenging. With reference to the above classification, features of the app are defined as the ability to detect objects in the surrounding space, receive descriptions of scenes, recognize colors, locate specific items, identify currency denominations, identify facial expressions, as well as to facilitate effective communication and improve the capacity to solve routine activities in daily life.[6].

1.1 Existing Solutions

To address the challenges faced by visually impaired individuals, several applications have been developed that leverage artificial intelligence and assistive technologies. However, these existing solutions often fail to provide a comprehensive, affordable, and fully independent system. For instance, **Be My Eyes** connects visually impaired users with volunteers for live video assistance, but it heavily depends on the availability of volunteers, raises privacy concerns due to live video sharing, and often encounters language barriers, limiting its usability. Similarly, **Tap Tap See** provides object identification through image processing but lacks real-time assistance and suffers from delays in delivering results, reducing its effectiveness in dynamic environments. **KNFB Reader**, while effective for text recognition and reading, is restricted to this single functionality, offering no support for object detection, scene comprehension, or navigation. Meanwhile, **Aira** provides real-time assistance via human agents but requires a costly subscription, which makes it

inaccessible to many users, and its reliance on human assistance undermines user independence.

Empowered AI surpasses these limitations by integrating multiple advanced features such as real-time object detection, detailed scene understanding, text-to-speech conversion, user-friendly, and cost-effective mobile application. Unlike its competitors, it offers a comprehensive, autonomous solution that enhances the independence and quality of life for visually impaired individuals.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Be My Eyes	Be My Eyes connects visually impaired users with volunteers through live video calls for real-time assistance. The app leverages human support for object recognition, navigation, and other tasks.	Relies on volunteer availability, which may cause delays. Privacy concerns arise due to live video streaming. The app is not fully autonomous and can struggle with language barriers.
Tap Tap See	Tap Tap See enables visually impaired users to take pictures of objects and receive verbal descriptions using image recognition technology.	Lacks real-time assistance. Image processing often suffers from delays, limiting the app's use in dynamic environments. Only provides basic object recognition.
KNFB Reader	KNFB Reader is a text recognition app that converts printed text into speech, enabling visually impaired users to read documents, books, and signs.	Limited to text recognition. Does not offer features such as object detection, scene understanding, or navigation assistance. Offers no real-time dynamic support.
Aira	Aira provides real-time assistance via trained human agents who guide visually impaired users through various tasks using a smartphone camera.	Requires a subscription service, making it expensive. Relies on human agents, limiting user autonomy. Offers no offline functionality.

1.2 Problem Statement

People with vision impairment experience multiple problems in mobility and management of everyday tasks. This results in making them develop a feeling of dependency, making them feel that they cannot do things on their own, thus a feeling of restricted freedom. It can be challenging to understand and interpret gestures and other nonverbal signs essential for interpersonal communication, which results in a sensation of loneliness or misinterpretation of other people's intentions and messages in social interactions.

However, the price of the existing assistive technologies can also be another challenge ahead. Some of these tools are very costly and unaffordable to a huge number of the visually impaired especially those in low-income areas. While they are still affordable, some of these devices are not adequate in offering complete solutions to execute complex tasks, as they usually focus on the particular issues that need to be solved.

Another limitation is that current assistive technologies are not fully compatible with each other, which causes the users to have to use a variety of gadgets or programs in order to fulfill their requirements during the day.

1.3 Motivation

The motivation to proceed with the **“Empowered-AI”** project comes from our passion for developing applications based on high-end technologies in computer vision and artificial intelligence as well as the desire to provide people with the impaired vision with more freedom and ability to rely on their own in many aspects of life. The visually impaired individuals face many challenges in going about their daily lives, which include mobility, understanding facial expressions, and being highly dependent on the support of other people.

Thus, our goal was to design a project that would not only let us develop and learn in the framework of such dynamically growing fields but also produce a practical result with the capability of improving people's lives. The goal of our work, therefore, was not only to create a tool that could be implemented as part of a purely academic setting but one that would present real-world advantages that might be useful for many years down the line.

1.4 Problem Solution

The **“Empowered-AI”** project will attempt to address the issues related to people with visual impairments by designing an all-in-one accessibility application that is based on AI advancements. It is worth underlining that this application is focused on improving the

independence of visually impaired users and contains useful tools and features that may help to work with the main problems stated in the problem section.

Objectives:

1. **Real-Time Object Detection:** Use an artificial intelligence approach for object recognition to help users have a vision of objects in their environment, allowing them to successfully avoid or interact with them safely.
2. **Scene Descriptions:** Exploit computer vision to provide a detailed account of the physical surrounding environment to the user and assist the user in the comprehension of the immediate world.
3. **Facial Emotion Recognition:** Further incorporate the facial recognition algorithm to capture and analyze the emotional gestures, helping the users learn social signals and, in extension, to enhance interpersonal interaction.
4. **Voice Feedback:** Incorporate voice feedback to relay information about objects, scenes, and emotions, allowing users to receive auditory information that complements visual and tactile inputs.
5. **Text Recognition:** Develop optical character recognition (OCR) capabilities to enable users to read printed or handwritten text, such as labels, signs, and documents, directly through the app.
6. **Color Recognition:** Implement a color detection feature that can identify and name colors in real-time, assisting users with tasks such as clothing coordination, distinguishing labeled objects, or recognizing traffic signals.
7. **Item Locator:** Incorporate a targeted item-finding feature to help users locate commonly used personal items like keys, wallets, or medication using visual or spatial cues.
8. **Currency Detection:** Add a currency recognition tool that identifies and announces denominations of paper money, supporting financial independence and secure transactions.
9. **Integration of Haptic Feedback:** Use haptic feedback to offer tactile responses that complement auditory information, improving the overall user experience and accessibility.
10. **Affordability and Accessibility:** Ensure the application is free of cost.
11. **User-Friendly Interface:** Design a simple, accessible interface with voice feedback to help visually impaired users easily navigate and use the app.

1.5 Scope

The "**Empowered-AI**" project is an application designed to enhance the independence of visually impaired individuals by providing essential tools for daily tasks. The app utilizes advanced AI technologies to deliver key functionalities such as real-time object detection, enabling users to identify and locate objects in their surroundings. It also produces scene descriptions through computer vision, which helps users comprehend their surroundings. Another significant aspect of this design is facial emotion recognition, indicating the definiteness of emotions and allowing the user to understand social signals. Furthermore, it has the OCR feature for the identification and scanning of printed or handwritten text within the app.

The app also introduces new capabilities such as color recognition, enabling users to distinguish between colors for clothing, food packaging, or other visual cues; item locator, which helps in locating misplaced objects within the environment; and currency detection, to ensure users can independently identify and handle paper money.

Specifically, voice feedback is utilized to convey all this information to users in order to create a seamless auditory experience.

This app is designed and located so that it is downloadable without cost to the users especially those in the low-income regions. In so doing, the project provides functionalities in one platform hence differing from the use of separate devices or costly assistive technologies. The Empowered-AI application needs to be created so that the visually impaired person is confident and independent with the help of the application.

Our project will not function effectively in night conditions. Although our model can detect objects similarly to the human eye at night, for clear detection and scene understanding, a night vision camera will be required, which is beyond the scope of this project.

1.6 Modules

Module 1: Vision Insights

- Data Set Analysis
- Object Detection Model Training
- Scene Understanding
- Color Recognition

Module 2: Smart Interface

- UI Figma Design
- Scene Description
- Text-to-Speech Integration
- Currency Detection

Module 3: Live Assist

- App Development
- Text Recognition
- Voice Feedback
- Item Locator

Module 4: Emotion Sense

- Facial Emotion Detection
- Testing and Quality Assurance

Module 5: Smart Assist Enhancements

- Item Locator
- Color Detection
- Currency Detection

1.7 Work Division

For each module and respective feature, assign responsibility to a team member.

Table 1.2: Team Member Responsibilities

Name	Registration	Responsibility/ Module / Feature
Mr. Abdul Wasay	21i-0834	Module 1 - Feat 1 Module 2 - Feat 1 Module 3 - Feat 1 Module 4 - Feat 1
Mr. Hamza Tariq	21i-0707	Module 1 - Feat 2 Module 2 - Feat 2 Module 3 - Feat 3 Module 4 - Feat 2
Mr. Hassan Abbas	21i-0507	Module 1 - Feat 3 Module 2 - Feat 3 Module 3 - Feat 2 Module 4 - Feat 2

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Event Response Table

Table 2.1 presents the different interaction events between the user and the system, along with the corresponding system states and responses. These responses are designed to provide clear, immediate feedback—either through speech or vibration—ensuring a smooth and accessible user experience. This table forms the basis for understanding the functional behavior of the app during various use cases.

Event	System State	Response
User opens the app	The welcome screen is displayed, indicating readiness for interaction.	The app reads aloud, "Welcome to Empowered-AI."
User issues voice command for Object Detection	The app transitions to Object Detection mode.	The app reads aloud, "Object Detection activated. The app is now detecting objects through the camera."
User activates Scene Description	The app prepares to provide a description of the current scene.	The app reads aloud, "Scene Description activated. Please capture an image."
User issues command for Text Recognition	The app prepares to recognize printed or handwritten text.	The app reads aloud, "Text Recognition activated. The app will now read the text in front of you."
User points camera at a person	The app processes the camera input to detect the person's emotional expression.	The app reads aloud, "Emotion Detection activated. The app is analyzing the person's emotional expression."
User interacts with an object through voice command	The app receives and processes the voice command for interaction.	The app vibrates to indicate a successful interaction.
User closes the app	The app prepares to exit and complete the session.	The app reads aloud, "Thank you for using Empowered-AI. Good-bye!"

Table 2.1: User Interaction Events with System Responses

2.2 Functional Requirements

This section describes the functional requirements of the system, organized by each module and its respective features.

2.2.1 Module 1: Vision Insights

Following are the requirements for module 1:

1. FR1: The system shall be able to detect and identify objects in real-time within the user's environment.
2. FR2: The system shall provide scene descriptions based on object detection and contextual understanding.

2.2.2 Module 2: Smart Interface

Following are the requirements for module 2:

1. FR1: The system shall describe scenes in a way that visually impaired users can understand via voice feedback.
2. FR2: The system shall integrate text-to-speech technology to facilitate easy interaction.

2.2.3 Module 3: Live Assist

Following are the requirements for module 3:

1. FR1: The system shall provide real-time assistance by recognizing printed and handwritten text using OCR.
2. FR2: The system shall provide voice feedback to users based on the text detected by the app.

2.2.4 Module 4: Emotion Sense

Following are the requirements for module 4:

1. FR1: The system shall detect facial emotions in real-time using AI-based emotion recognition technology.
2. FR2: The system shall test all functionalities to ensure they provide accurate and reliable outputs.
3. FR3: The system shall conduct thorough quality assurance to ensure robustness across different environments.

2.2.5 Module 5: Smart Assist Enhancements

Following are the requirements for module 4:

1. FR1: The system shall assist users in locating personal items (e.g., keys, wallets) using real-time object detection and targeted search.
2. FR2: The system shall detect and identify colors in real-time and provide immediate voice feedback for recognized colors.
3. FR3: The system shall recognize different currency denominations from images and audibly announce the detected value to the user.

2.3 Non-Functional Requirements

This section specifies non-functional requirements. These quality requirements should be specific, quantitative, and verifiable.

2.3.1 Reliability

- RELIABILITY-1: The system shall maintain a mean time between failures (MTBF) of at least 168 hours of continuous operation.
- RELIABILITY-2: The application shall have a maximum allowable downtime of no more than 1 day per month for maintenance and updates.

2.3.2 Usability

- USABILITY-1: The application shall allow users to access main features (object detection, scene description, text recognition, etc.) within three taps.

- **USABILITY-2:** New users shall be able to learn core functionalities of the app within 30 minutes of use.
- **USABILITY-3:** The application shall provide clear and concise audio instructions for all features.

2.3.3 Performance

- **PERFORMANCE-1:** 90% of scene descriptions shall be processed and relayed to the user within 15 seconds of input.
- **PERFORMANCE-2:** Voice feedback responses shall have a latency of no more than 3 seconds.

2.3.4 Security

- **SECURITY-1:** The system shall ensure all data is processed securely in-memory and discarded immediately after use.
- **SECURITY-2:** The application shall not store any personal or sensitive information locally or on external servers.

Chapter 3

System Overview

3.1 Architectural Design:

A **client-server architecture** fits well as it allows for the seamless delivery of complex AI functionalities while ensuring scalability and performance. The app (client) installed on users' devices can communicate with powerful servers to process intensive tasks like object detection, facial emotion recognition, and scene understanding using machine learning models. By offloading these computations to the server, The server handles the AI processing and returns real-time results, while the client provides immediate audio feedback, creating an efficient, responsive, and scalable solution for all users.

3.1.1 Explanation of Architecture Diagram:

In the architecture diagram, mobile clients send data from the environment, such as images or real-time streams, to a server through a network interface. The server performs tasks like object detection, text recognition, scene description, and facial emotion detection using various AI models. Once the server processes the data, the results are sent back to the mobile client. Through voice feedback represented by the 'loudspeaker' symbol, the user receives the processed data which can be scenes description, text recognition or feelings analysis. This architecture can distribute the computing burden across the server in an efficient but real time manner required for mobile applications accessibility to the visually impaired.

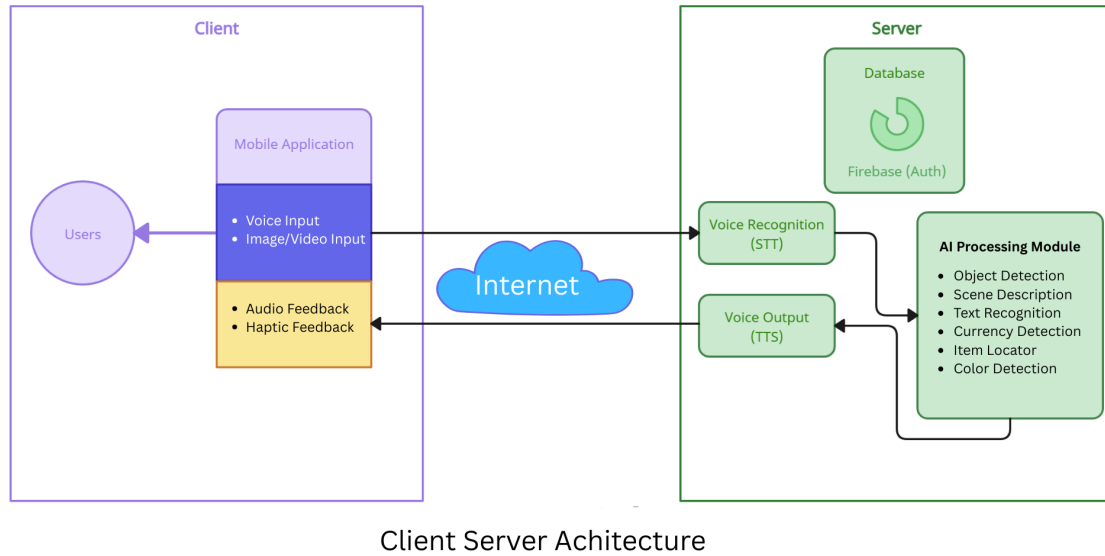


Figure 3.1: Architecture Diagram

enumitem

3.2 Data Design

This section describes the data flow and storage design for a mobile application focused on image and video processing. The following explains each component and its interactions.

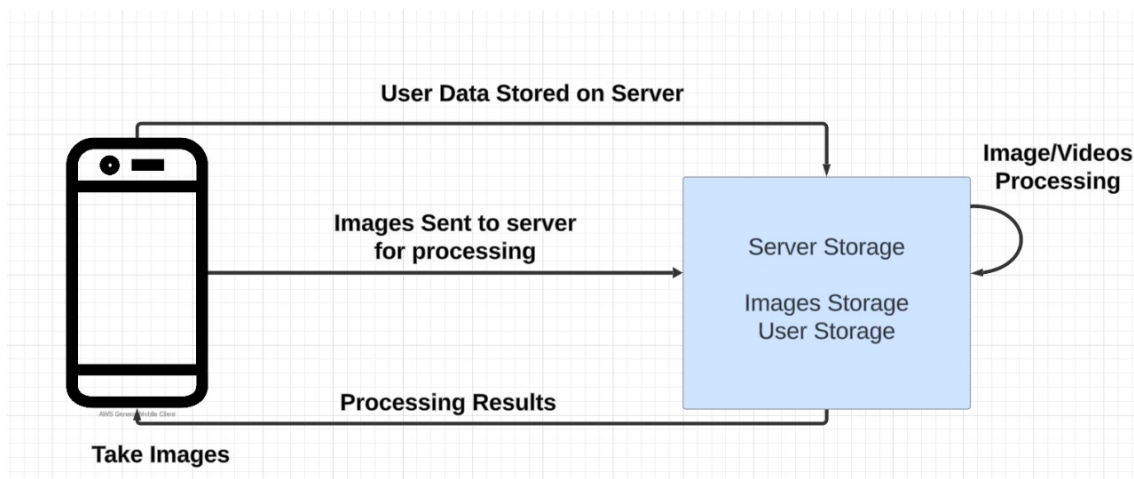


Figure 3.2: Data Design Diagram

3.2.1 Mobile Device

- **Take Images:** This step represents the user capturing images or videos using the application on their mobile device.
- The mobile device acts as the source of data (images or videos) to be processed.

3.2.2 Image Transmission

Images Sent to Server for Processing: After capturing, images are transmitted from the mobile device to the server. This typically involves secure upload mechanisms such as APIs or cloud services.

3.2.3 Server Processing and Storage

- **Server Storage:** Responsible for:
 - Storing user data.
 - Storing uploaded images and videos.
- **Image/Video Processing:** The server hosts processing units that perform tasks such as analysis, enhancement, or transformation. This may include AI-based algorithms, filters, or other functionalities.
- **User Storage:** Maintains a structured repository of user-specific data, including processed results and related records.

3.2.4 Results Transmission

Processing Results: After processing, the server sends results back to the mobile device, which may include:

- Enhanced images or videos.
- Processed data such as insights or generated reports.

3.2.5 User Data Storage Feedback

User Data Stored on Server: Processed data and user-related information are stored for long-term access, allowing users to retrieve their data without re-uploading or reprocessing.

3.2.6 Overall Workflow

1. **Input:** Users capture images or videos through the application.
2. **Upload:** Raw media files are transmitted to the server.
3. **Processing:** The server processes and stores data in the appropriate storage systems.
4. **Result Delivery:** Processed results are sent back to the user's device.
5. **Storage:** All user-related data, raw and processed, are securely stored on the server for future access.

3.2.7 Database Selection

The databases used for this project are:

- **Firebase**
- **MongoDB**

3.3 Domain Model

The domain model of the "**Empowered AI**" application represents a system designed to assist users, particularly visually impaired individuals, by providing various functionalities like object detection, scene description, emotion recognition, text recognition, and voice feedback. Below is the Domain model along with breakdown of the entities, their attributes, and the relationships between them.

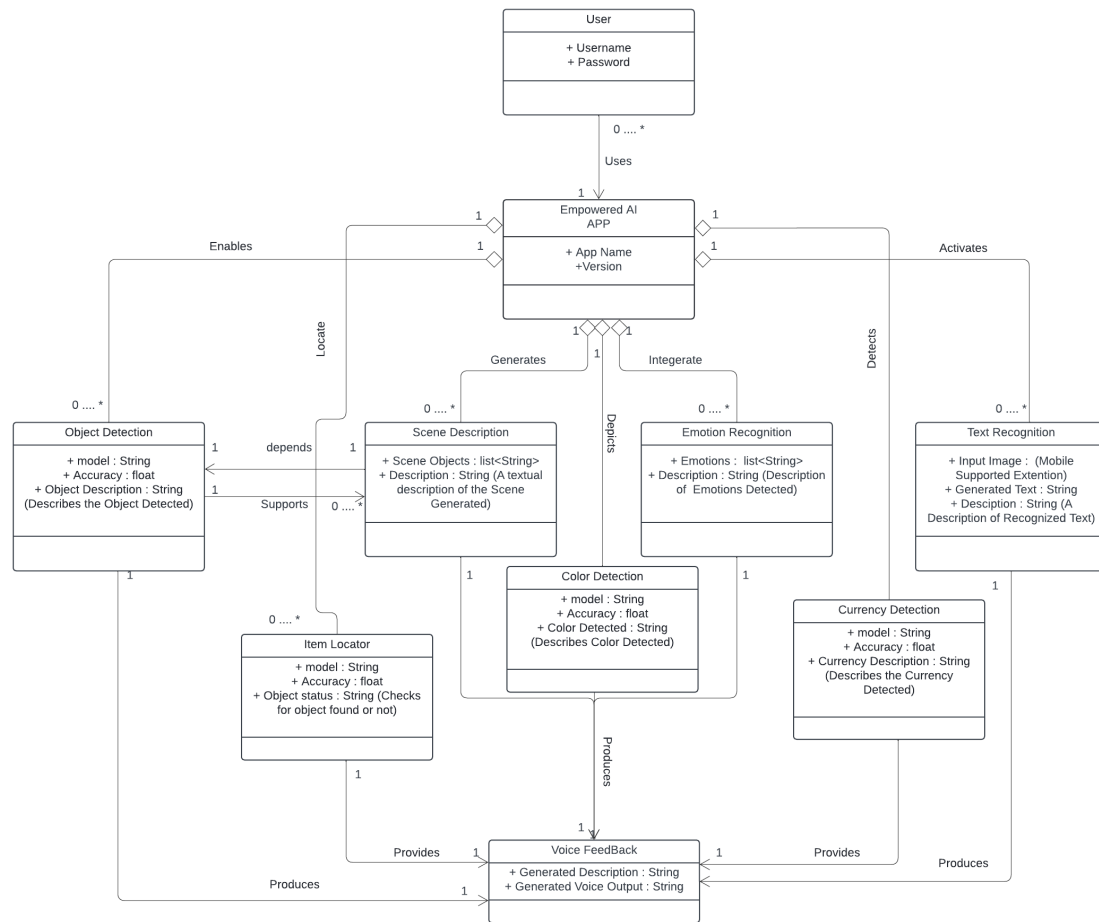


Figure 3.3: Domain Model

3.3.1 Entities and Their Attributes

1. User:

- Represents the individual interacting with the Empowered AI application.
- Attributes:
 - **Username:** String (User's login name).
 - **Password:** String (User's password for authentication).

2. Empowered AI App:

- The central entity that integrates and activates all other modules.
- Attributes:
 - **App Name:** String (Name of the application).

- **Version:** String (Version number of the app).

3. Object Detection:

- Detects objects in the user's environment.
- Attributes:
 - **Model:** String (AI model used for object detection).
 - **Accuracy:** Float (Confidence score of the detection).
 - **Object Description:** String (Description of the detected object).

4. Scene Description:

- Provides a description of the surrounding environment based on detected objects.
- Attributes:
 - **Scene Objects:** List of Strings (Objects detected in the scene).
 - **Description:** String (Textual description of the scene).

5. Emotion Recognition:

- Identifies emotions based on input data such as facial expressions.
- Attributes:
 - **Emotions:** List of Strings (Detected emotions like happy, sad, etc.).
 - **Description:** String (Detailed description of the detected emotions).

6. Text Recognition:

- Extracts text from images or scanned documents.
- Attributes:
 - **Input Image:** Image (Captured image containing text).
 - **Generated Text:** String (Extracted text).
 - **Description:** String (Description of the recognized text).

7. Audio Response:

- Converts detected information into auditory response for the user.
- Attributes:
 - **Generated Description:** String (Summary of detected objects, scenes, or emotions).
 - **Generated Voice Output:** String (Audio output for user interaction).

3.3.2 Relationships

1. **User → Empowered AI App**: A **User** activates and uses the **Empowered AI App**.
2. **Empowered AI App → Object Detection**: The app **enables** object detection, allowing the identification of objects within the user's environment.
3. **Object Detection → Scene Description**:
 - **Object Detection** is a prerequisite for generating a **Scene Description**.
 - The relationship: **Object Detection depends on Scene Description** to provide a comprehensive description of the environment.
4. **Scene Description → Emotion Recognition**:
 - **Scene Description** integrates data from **Emotion Recognition** to provide emotional context in the scene.
 - **Emotion Recognition provides** emotional feedback to the **Scene Description**.
5. **Empowered AI App → Text Recognition**: The **Empowered AI App** activates **Text Recognition**, enabling it to extract information from images.
6. **Text Recognition → Audio Response**:
 - The text recognized by the **Text Recognition** module is passed to **Voice Feedback**, which converts it into audible output.
 - Relationship: **Text Recognition produces** information for **Voice Feedback**.
7. **Scene Description & Emotion Recognition → Audio Response**:
 - Both **Scene Description** and **Emotion Recognition** provide input to **Audio Response**, which generates auditory descriptions for users.
 - Relationship: **Audio Response integrates** data from multiple sources.

3.3.3 Cardinalities

- **One-to-One**:
 - Each **User** interacts with one instance of the **Empowered AI App**.
 - Each **Text Recognition** input corresponds to a single **Generated Text** output.
- **One-to-Many**:

- **Scene Description** can include multiple **Scene Objects**.
- **Audio Response** may generate audio output from multiple input sources like text, objects, and emotions.

An example of a domain model for an online shopping system is provided in Appendix B.

3.4 Design Models

For this purpose, in our project, We proposed to follow Object-Oriented Development Approach (OOP) as to advantages in complexity management We identified that our system is highly complex and comprises a number of elements, including object detection, scene description, text recognition, and emotional recognition. The use of OOP we can format these components as distinct objects. This modularity helps in code accountability that is, developing, testing and experiencing each part of the system free from the influence of the other. To the credit of this concept it supports our goals of designing an extensible and sustainable application – factors that make the total development more manageable and the final product stronger.

3.4.1 Activity Diagram

3.4.1.1 Activity Diagram 1: Object Detection

The Visually impaired user launches the EmpoweredAI app and uses voice to make the application perform an object detection. It launches the camera and uses it to record a video of the environment, the video is then streamed to the server for the objects to be recognized before converting it to an audio format. It also narrates descriptions of the detected objects so that the user can have a basic idea about the environment.

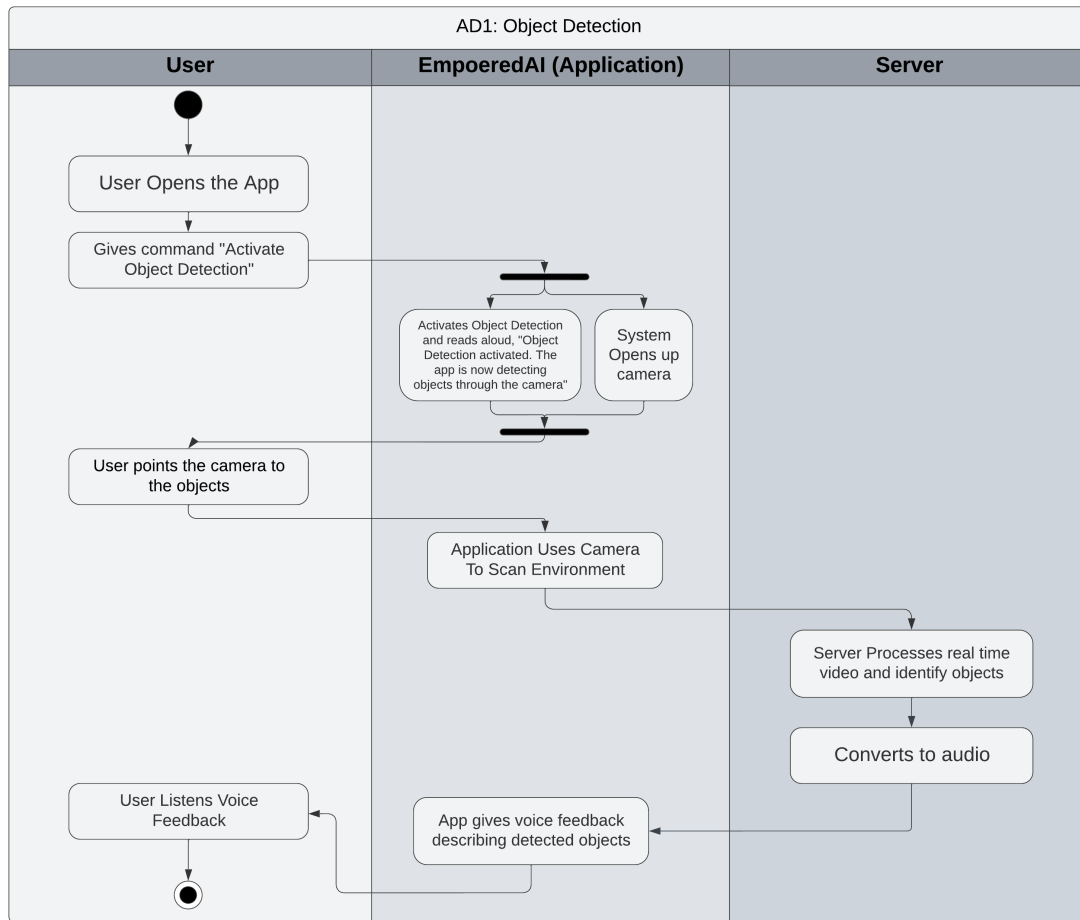


Figure 3.4: Object Detection Activity Diagram

3.4.1.2 Activity Diagram 2: Scene Description

The user launches EmpoweredAI application and utters a voice command to start the so-called “Scene Description” which asks the user to take a picture. The described image is transferred to the server, processed there by special algorithms and converted into an audio signal; the application gives an instant voiced commentary to the image for the user to listen.

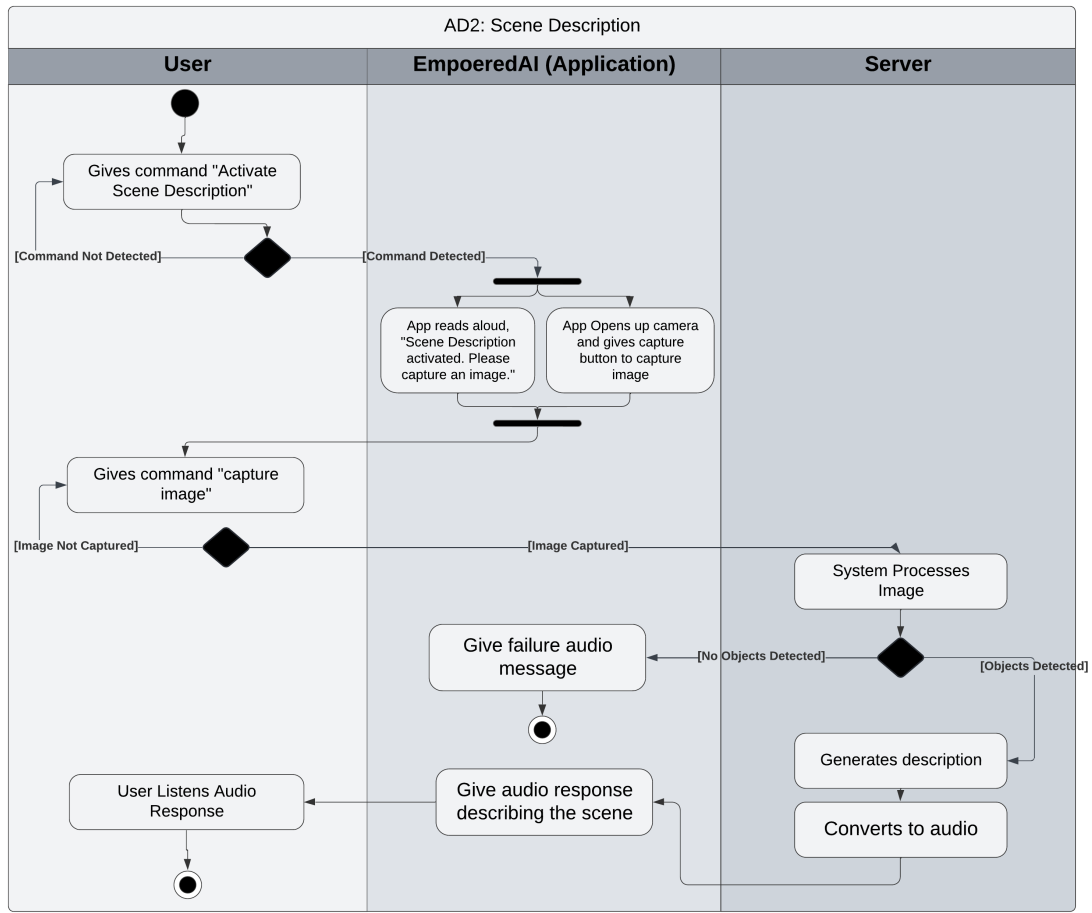


Figure 3.5: Scene Description Activity Diagram

3.4.1.3 Activity Diagram 3: Text Recognition

The user launches EmpoweredAI application and utters a voice command to start the so-called “Text Recognition” which asks the user to take a picture. The described image is transferred to the server, processed there by special algorithms and converted into an audio signal; the application gives an instant voiced commentary to the for the text from the image for the user to listen.

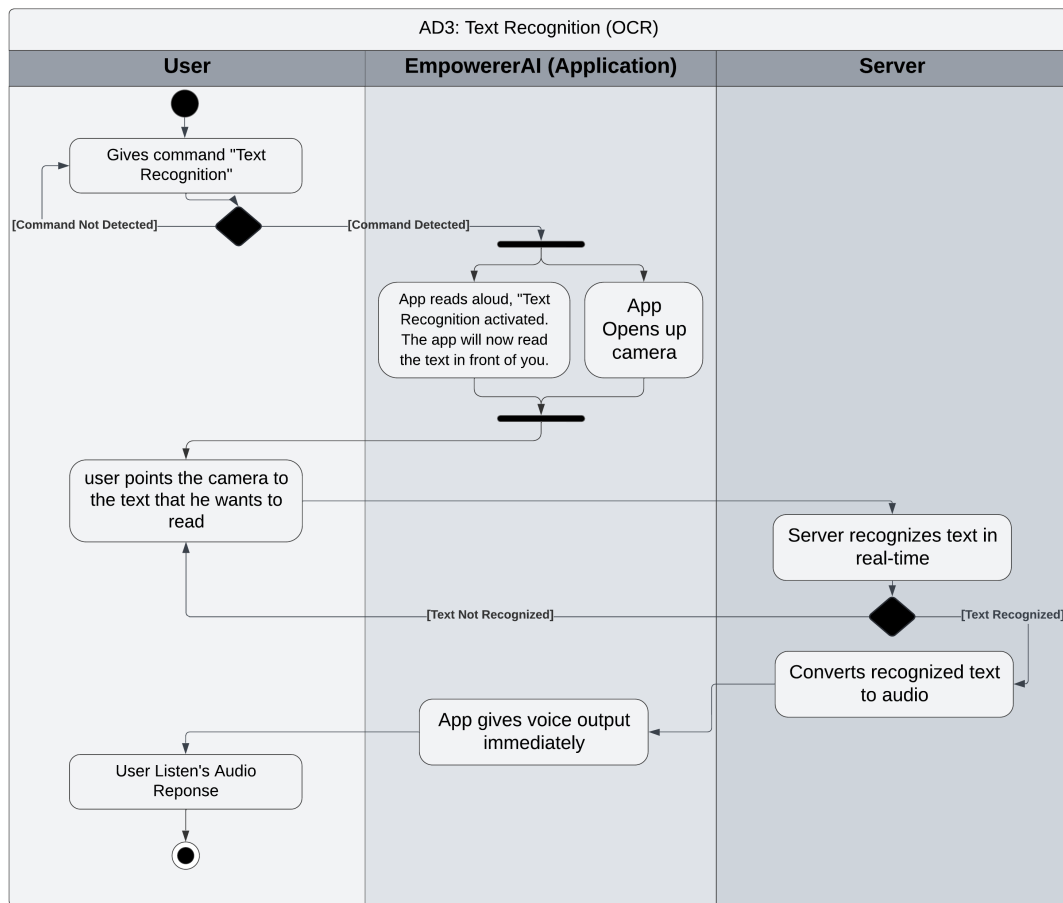


Figure 3.6: Text Recognition Activity Diagram

3.4.1.4 Activity Diagram 4: Emotion Detection

The user launches EmpoweredAI application and utters a voice command to start the so-called "Emotion Detection" which asks the user to take a picture. The described image is transferred to the server, processed there by special algorithms and converted into an audio signal; the application gives an instant voiced commentary according to the emotion detected of the person in the image for the user to listen.

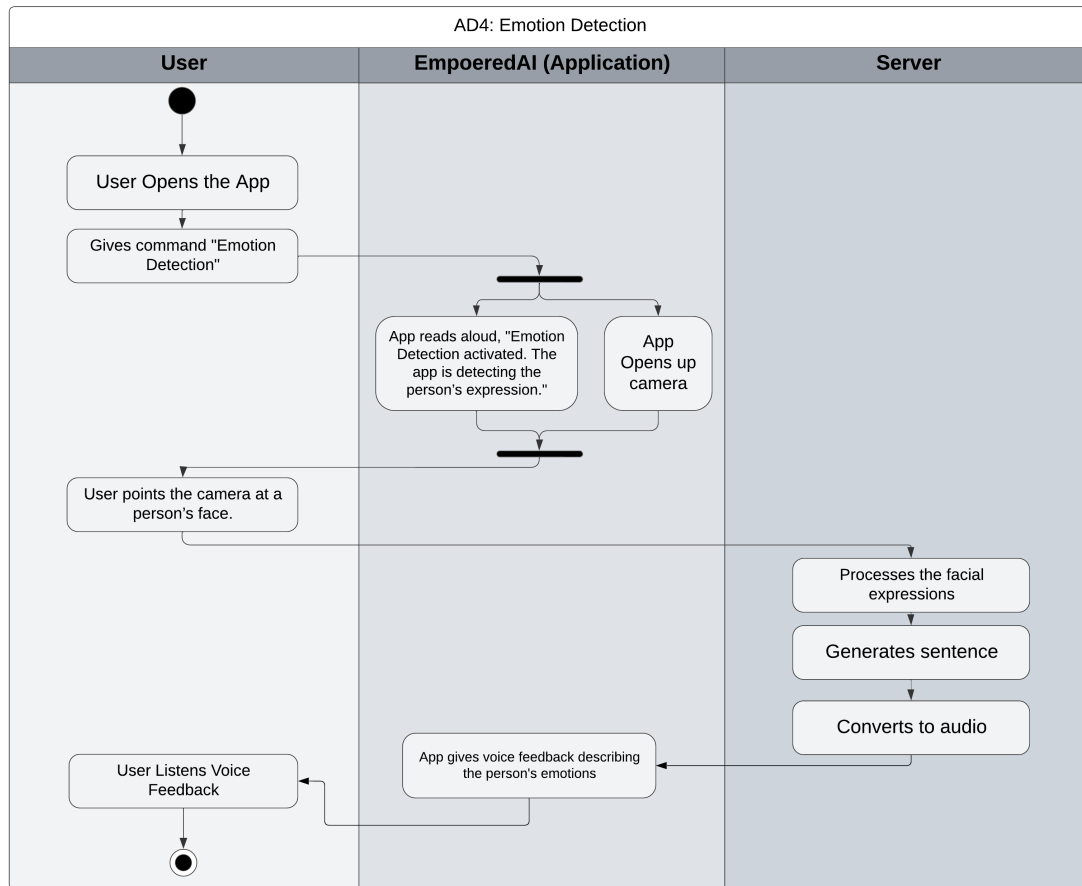


Figure 3.7: Emotion Detection Activity Diagram

3.4.1.5 Activity Diagram 5: Currency Detection

The user launches the EmpoweredAI application and utters a voice command to initiate the “Currency Detection” feature, which prompts the user to capture an image of the currency note. The captured image is then sent to the server, where it is processed using a self-trained model . The system identifies the note and converts the result into an audio message, providing the user with an immediate verbal announcement of the detected currency.

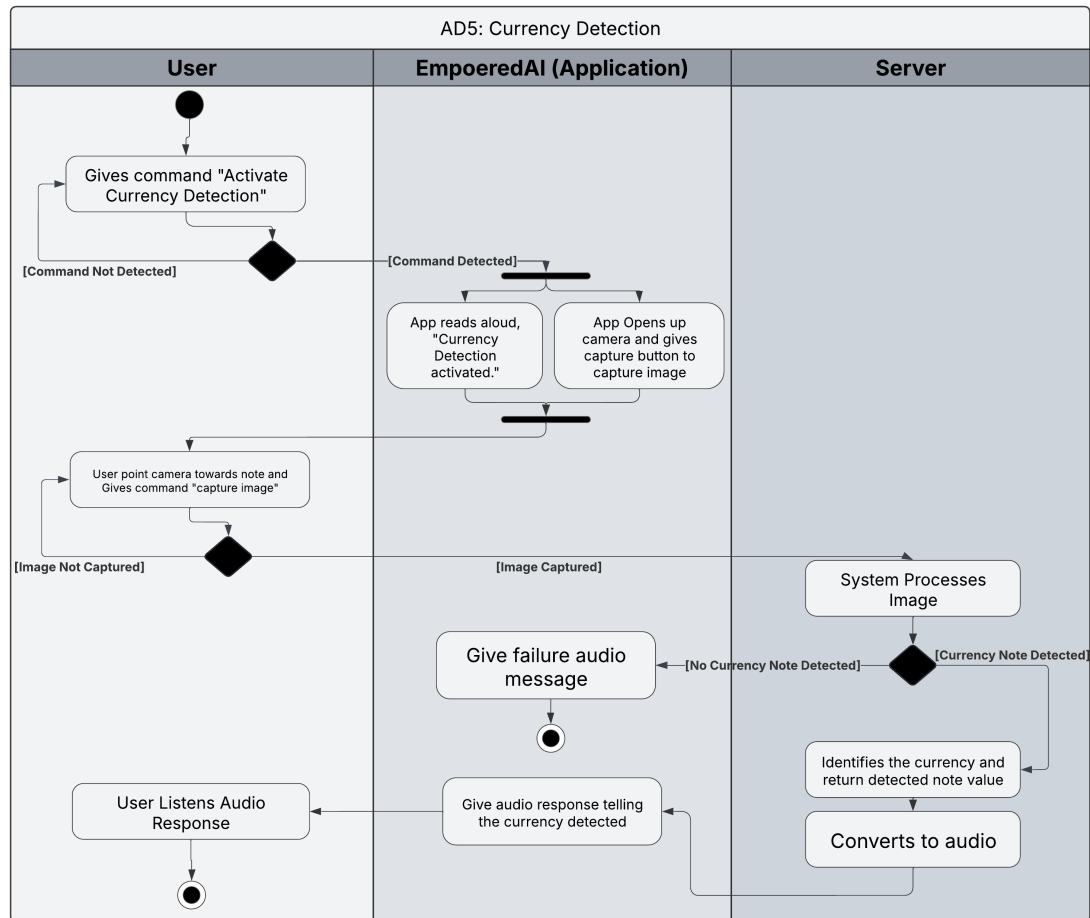


Figure 3.8: Currency Detection Activity Diagram

3.4.1.6 Activity Diagram 6: Item Locator

The user activates the “Item Locator” feature within the EmpoweredAI application. The system prompts the user to specify the object they are searching for via a voice command. Upon recognizing the object name, the application begins scanning the environment to locate it. As the system gets closer to the identified object, it emits a series of beeps with increasing frequency to guide the user toward the target. Additionally, users can expand the database of recognizable items by recording a video of a new object and assigning it a name, enabling personalized object detection in future searches.

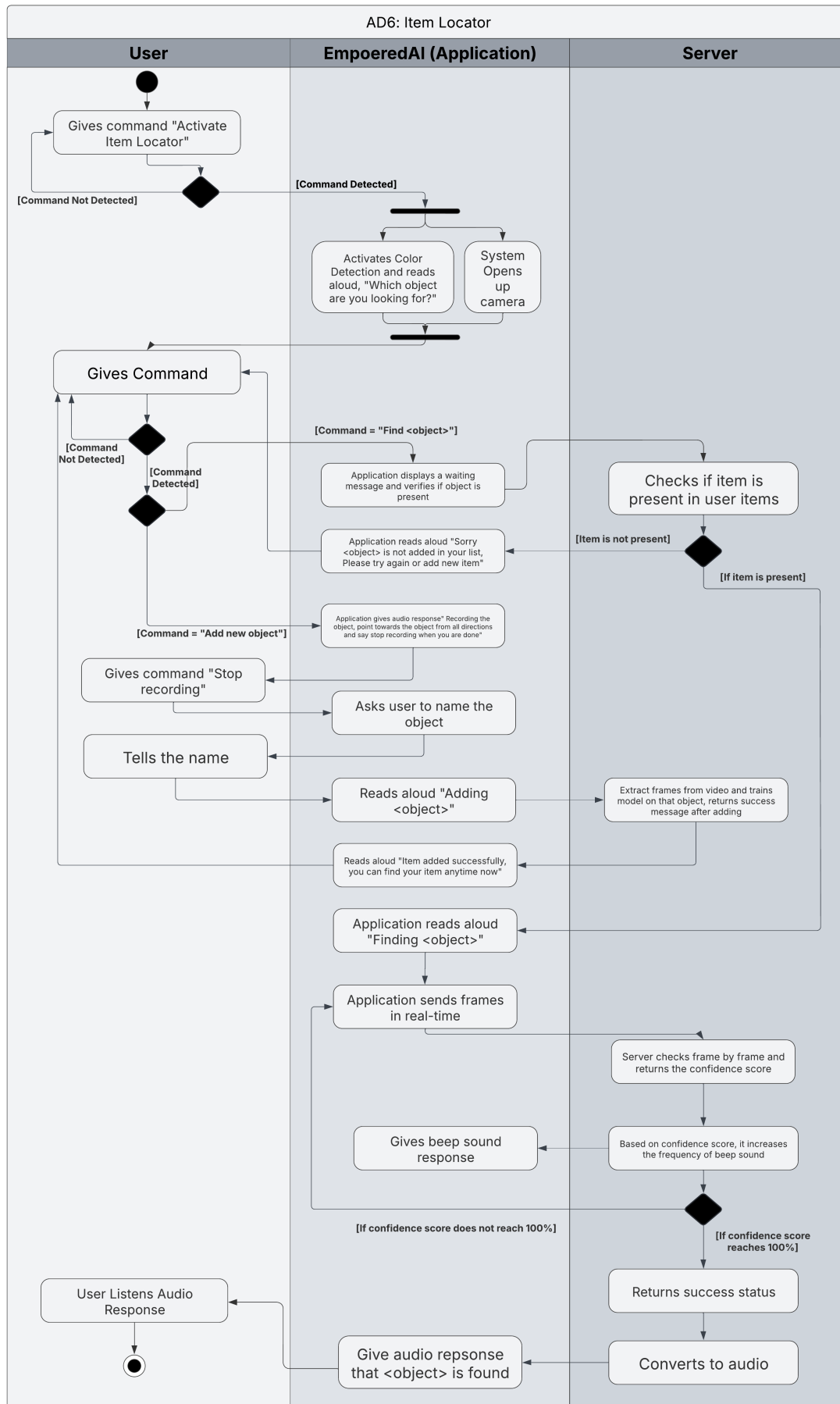


Figure 3.9: Currency Detection Activity Diagram

3.4.1.7 Activity Diagram 7: Color Detection

The user activates the “Color Detection” feature in the EmpoweredAI application. The system prompts the user to point the device’s camera at the desired location or object. The application analyzes the color at a specific point in the camera’s view—typically the center of the screen—and identifies the dominant color at that location. It then provides an immediate audio response announcing the detected color, assisting users, especially those with visual impairments, in identifying and differentiating colors in their environment.

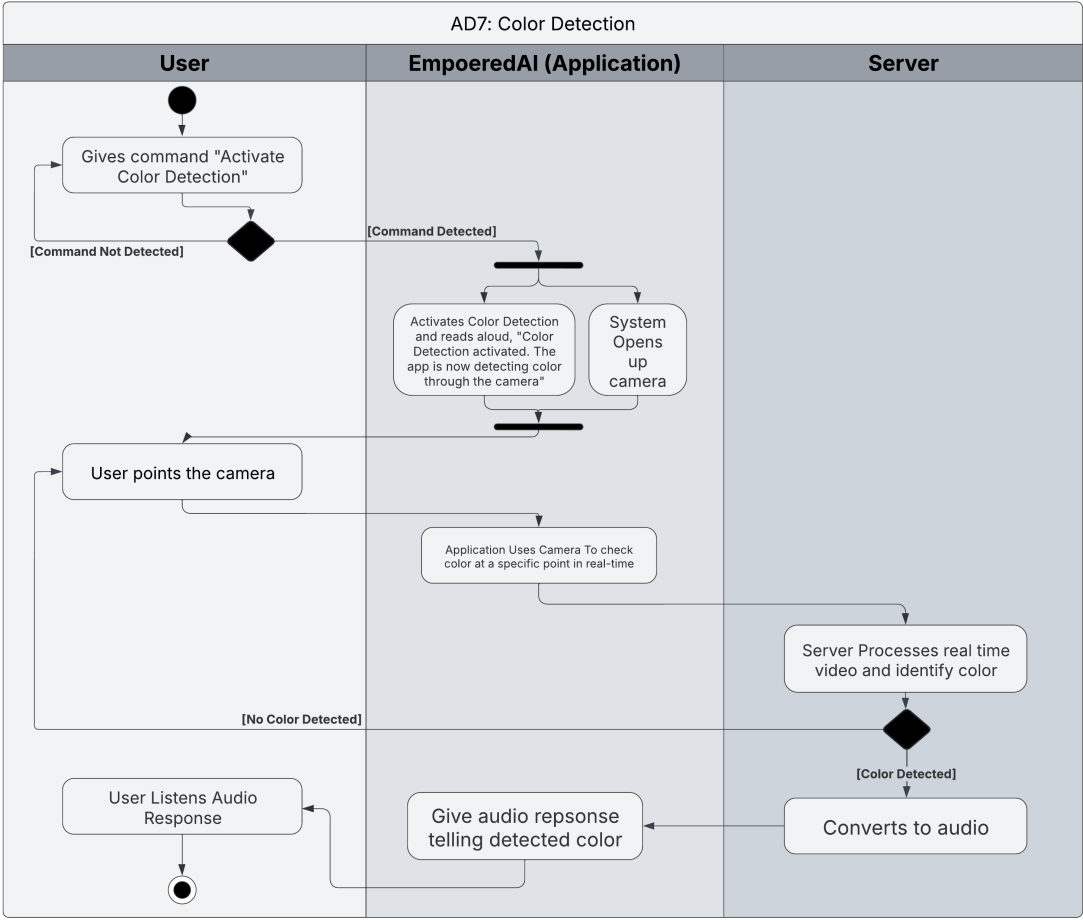


Figure 3.10: Color Detection Activity Diagram

3.4.2 Class Diagram

The class diagram represents the architecture of the **Empowered AI App**, showcasing its primary components and the relationships among them. This system integrates various subsystems to provide object detection, scene description, emotion recognition, text

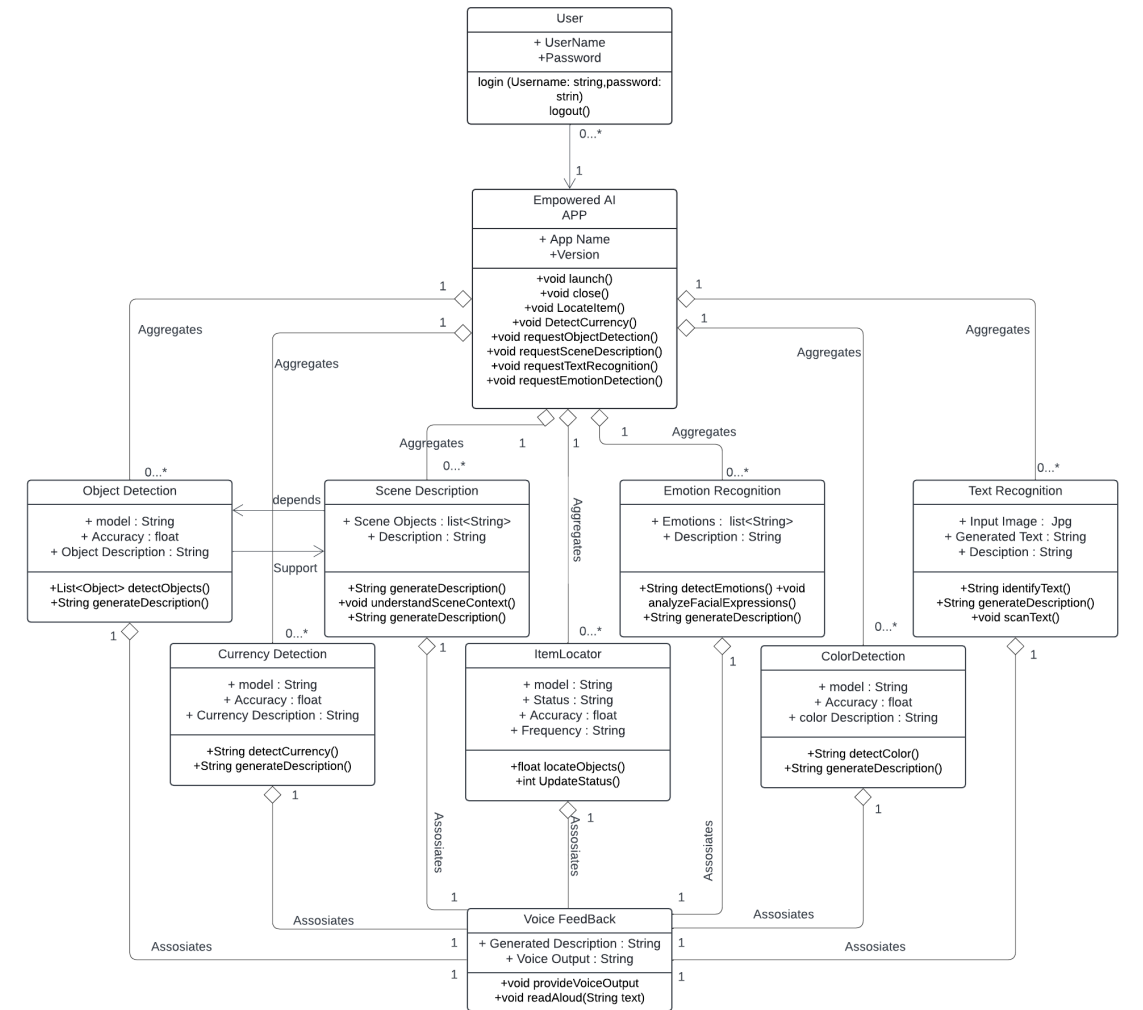


Figure 3.11: Class Diagram

1. User Class

- Represents the app users.
- Attributes:
 - Username: Identifies the user.
 - Password: Authenticates the user.
- Methods:
 - `login(username: String, password: String): Boolean`: Authenticates the user and grants access to the app.

- `logout()`: Logs the user out of the application.
- **Relationship**: Associates with the **Empowered AI App**, allowing users to interact with the system.

2. Empowered AI App Class

- Serves as the central component, coordinating interactions between users and subsystems.
- Attributes:
 - App Name: The name of the application.
 - Version: The current version of the app.
- Methods:
 - `playScenario(): String`: Demonstrates the app's functionality by combining subsystem outputs.
 - `associateWith(user: User)`: Links the app's operations to a specific user.
- **Relationships**: Aggregates with other components like **Object Detection**, **Scene Description**, **Emotion Recognition**, **Text Recognition**, and **Audio Response**.

3. Object Detection Class

- Implements object detection using a pre-defined model.
- Attributes:
 - model: Specifies the object detection model used.
 - Accuracy: Indicates the accuracy of detection.
 - Object Description: Provides a textual summary of detected objects.
- Methods:
 - `detectObjects(image: InputImage): List<ObjectDescriptions>`: Processes an input image to identify and describe objects.
 - `getDetectionAccuracy(): Float`: Returns the accuracy of the detection process.
- **Relationship**: Depends on the **Scene Description** class, providing detected objects for scene generation.

4. Scene Description Class

- Creates a comprehensive description of the detected scene based on objects provided by **Object Detection**.
- Attributes:
 - Scene Objects: A list of objects in the scene.
 - Description: A textual representation of the scene.
- Methods:
 - generateSceneDescription(objects: List<String>): String: Produces a textual description of the scene.
 - getSceneObjects(): List<String>: Returns the list of objects in the scene.
- **Relationships:**
 - Supports **Emotion Recognition** by providing contextual scene data.
 - Associates with **Audio Response**, supplying scene descriptions for auditory output.

5. Emotion Recognition Class

- Identifies emotions based on facial expressions in the provided image.
- Attributes:
 - Emotions: A list of detected emotions.
 - Description: A textual explanation of identified emotions.
- Methods:
 - detectEmotions(image: InputImage): List<Emotion>: Analyzes the image to identify emotions.
 - generateEmotionDescription(emotions: List<Emotion>): String: Provides a textual description of the emotions detected.
 - getEmotionsLog(): List<String>: Logs the detected emotions.
- **Relationships:** Associates with **Audio Response**, supplying emotional context for auditory descriptions.

6. Text Recognition Class

- Recognizes text from input frames or images.
- Attributes:
 - `InputImage`: The image or video frame being analyzed.
 - `Supported Text`: Indicates the languages or text types supported.
 - `Generated Text`: The recognized text extracted from the image.
- Methods:
 - `recognizeText(image: InputImage): String`: Extracts textual content from the image.
 - `generateTextDescription(text: String): String`: Produces a summary of the recognized text.
- **Relationships**: Associates with **Audio Response**, providing text data for voice synthesis.

7. Audio Response Class

- Converts textual data from other subsystems into audible response.
- Attributes:
 - `Generated Description`: The combined description provided by other subsystems.
 - `Generated Voice Output`: The synthesized voice output for the user.
- Methods:
 - `generateVoiceOutput(description: String): Audio`: Converts the description into audio response.
 - `playVoiceFeedback(audio: Audio)`: Plays the audio feedback to the user.
- **Relationships**: Associates with **Scene Description**, **Emotion Recognition**, and **Text Recognition**, integrating their outputs into a unified auditory experience.

3.4.3 Class-level Sequence Diagram

The sequence diagram illustrates the interaction between the user and various modules of the "Empowered-AI" app, from opening the app to processing different AI-powered functionalities.

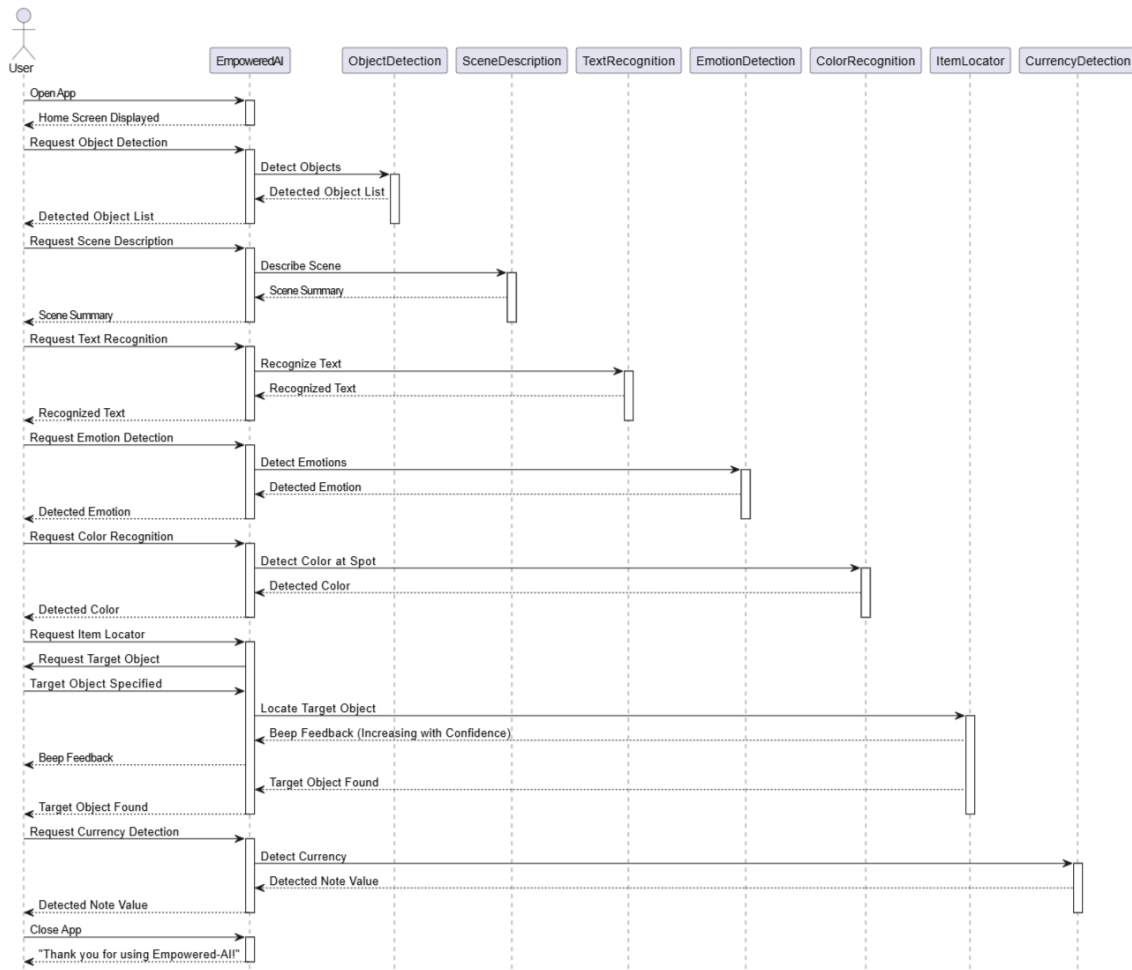


Figure 3.12: Class-Level Sequence Diagram

- **Open App:** The user opens the Empowered-AI app, which displays the home screen.
- **Request Object Detection:** The user requests object detection via a voice command, prompting the app to activate the ObjectDetection module to analyze the environment.
- **Provide Object Detection Result:** The app provides audio feedback, describing the detected objects.
- **Request Scene Description:** The user requests a scene description; the SceneDescription module processes and generates a description of the scene.
- **Provide Scene Description:** The app delivers the scene description as audio feedback to the user.
- **Close App:** The user closes the app, and a closing message is either displayed or

spoken.

3.4.4 State Transition Diagram

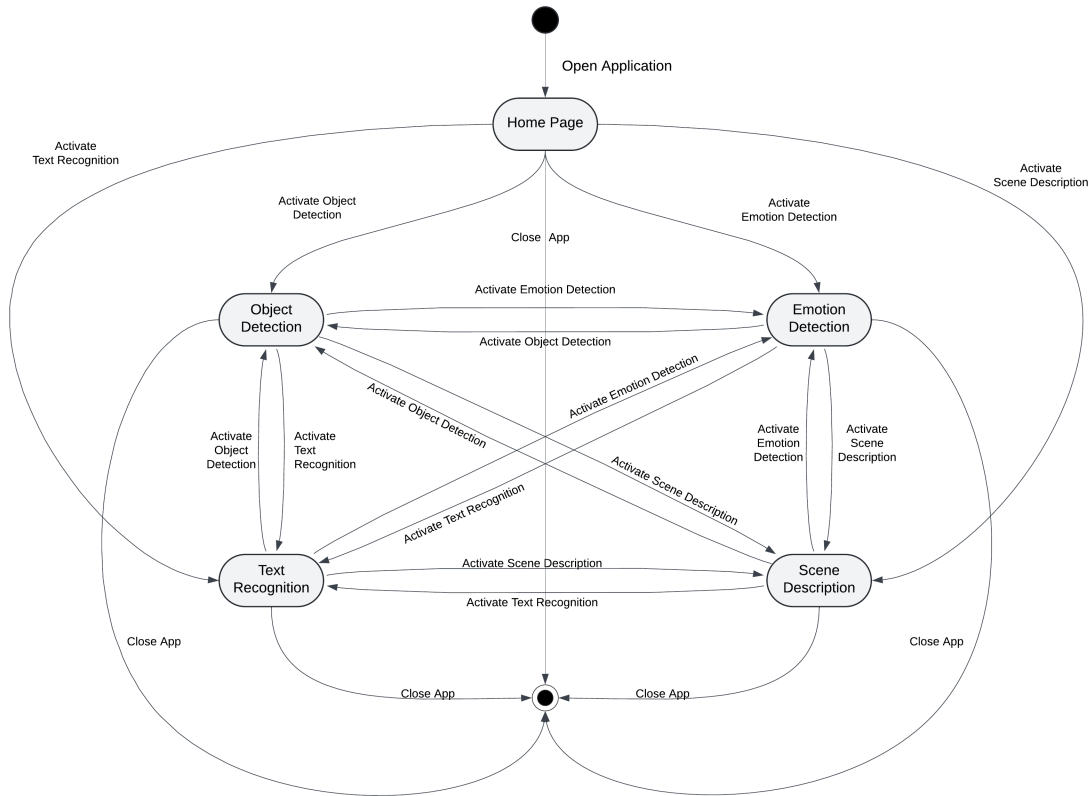


Figure 3.13: State Transition Diagram

The State Transition Diagram (Figure ??) illustrates the various states within the EmpoweredAI application and the transitions between them. The application begins at the "Home Page" state, from which the user can initiate various functionalities through corresponding voice commands. These include:

- **Object Detection:** Transition to this state occurs when the user activates object detection. Once object detection is completed, the user can either exit the application or activate another feature.
- **Scene Description:** The user can also activate scene description from the home page or from other states, generating a description of the captured scene.

From each of these functional states (Object Detection, Scene Description), the user can transition to other features or exit the application. Additionally, there is a direct transition to the **Close App** state from every functional state, allowing the user to exit the application at any point.

This diagram highlights the user's ability to seamlessly navigate between the various features of the EmpoweredAI app and the overall flow of interactions.

Chapter 4

Implementation and Testing

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

4.1 Algorithm Design

1. Object Detection Algorithm This algorithm enables the app to recognize objects in the user's surroundings using AI-powered object detection models. It identifies objects in real-time and provides auditory feedback to help users navigate safely and interact with their environment.

```
Algorithm ObjectDetection
Input: Real-time video stream from the user's device camera.
Output: Audio output with the names of detected objects.

1. Start
2. Initialize the camera and begin capturing video stream.
3. Load the pre-trained ObjectDetectionModel.
4. WHILE camera is active DO
5.     Capture current frame
6.     Send frame to ObjectDetectionModel
7.     DETECT objects in frame
8.     IF objects are detected THEN
9.         Generate descriptions of detected objects
10.        Convert descriptions to audio using Text-To-Speech (TTS)
11.        Play audio feedback for the user
12.    ELSE
13.        Notify user with "No objects detected"
14.    ENDIF
15. ENDWHILE
16. STOP
```

Figure 4.1: Object Detection Algorithm

2. Scene Description Scene description generates an overview of the surrounding environment by analyzing objects, their spatial relationships, and context. It offers users an auditory summary to help them comprehend their surroundings comprehensively.

```
Algorithm SceneDescription
Input: Single image captured by the user.
Output: Audio summary of the scene.

1. Start
2. Capture image from camera upon user command.
3. Send image to SceneUnderstandingModel on server.
4. Analyze objects, locations, and relationships.
5. Generate a textual scene description.
6. Convert the scene description to audio using TTS library.
7. Play audio description for the user.
8. Stop
```

Figure 4.2: Scene Description Algorithm

3. Text Recognition This module identifies and interprets textual information from the surrounding environment using optical character recognition (OCR) techniques. Recognized text is converted into audio output to assist users with visual impairments. Additionally, key terms or high-priority information trigger distinct vibration patterns to indicate importance, ensuring effective multi-sensory feedback and enhancing situational awareness.

```
Algorithm TextRecognition
Input: Image frame or user-captured image containing text.
Output: Audio playback of recognized text.

1. Start
2. Capture image or video frame upon user command.
3. Apply Optical Character Recognition (OCR) to the image.
4. If text detected:
    a. Extract and process text content.
    b. Convert text to audio using TTS library.
    c. Provide audio feedback of the text.
5. If no text detected:
    a. Notify user via audio ("No text found").
6. Stop
```

Figure 4.3: Text Recognition Algorithm

4. Facial Emotion This module processes captured images to identify and interpret facial expressions, determining emotions such as happiness, sadness, anger, or surprise. It provides auditory summaries of detected emotions. This assists users, particularly those

with visual impairments, in understanding the emotional context of interactions through non-visual cues.

```
Algorithm EmotionDetection
Input: Real-time video stream from camera focusing on faces.
Output: Audio feedback of detected emotions.

1. Start
2. Initialize camera and capture video stream.
3. For each frame in the video stream:
    a. Pass frame to FacialEmotionRecognitionModel.
    b. Detect facial expressions and emotions.
    c. If emotions detected:
        i. Generate descriptive text for the emotions.
        ii. Convert emotion description to audio.
        iii. Provide real-time audio feedback to the user.
    d. If no emotions detected:
        i. Notify user ("No emotions detected").
4. Repeat until user stops the feature.
5. Stop
```

Figure 4.4: Facial Emotion Algorithm

5. Audio Response This module integrates audio responses to deliver information from other functionalities. It translates detected objects, scenes, or emotions into auditory output while offering vibration patterns to signify event importance, ensuring multi-sensory feedback.

```
Algorithm VoiceAndHapticFeedback
Input: Textual information from AI modules.
Output: Real-time auditory and tactile responses.

1. Start
2. Receive input from ObjectDetection, SceneDescription, TextRecognition, or EmotionDetection.
3. Convert textual information to audio using TTS library.
4. Play audio output through device speakers.
5. If haptic feedback enabled:
    a. Generate vibration pattern corresponding to event importance.
6. Stop
```

Figure 4.5: Audio Response Algorithm

4.2 External APIs/SDKs

The following table lists and describes the third-party APIs and SDKs used during the development and implementation of the EmpoweredAI app:

API and Version	Description	Purpose of Usage	API Endpoint / Function / Class Used
Firebase (latest)	Backend-as-a-Service platform by Google	User authentication, storing user data, image uploads	FirebaseAuth, FirebaseStorage, Firestore
TensorFlow Lite Flutter Plugin	Lightweight runtime for deploying ML models on mobile devices	Run YOLO-based object detection model locally on device	tflite.loadModel(), tflite.runModelOnFrame()
Camera Package (latest)	Flutter plugin for accessing device camera in real-time	Capture live video feed for object detection and scene analysis	CameraController, CameraPreview
Image Picker (latest)	Flutter plugin to select images from gallery or capture via camera	Allow users to upload images for scene or text analysis	ImagePicker.pickImage()
Path Provider (latest)	Flutter plugin for accessing commonly used storage paths	Save inference outputs and intermediate data locally	getTemporaryDirectory(), getApplicationDocuments()

Table 4.1: List of External APIs and SDKs used in EmpoweredAI

4.3 Testing Details

EmpoweredAI current modules under implementation are being tested to identify any bugs and functionality issues. By performing manual unit testing, the working of each module is verified as intended.

4.3.1 Unit Testing

Each unit test is designed to test a specific function or method independently from other components, helping to identify issues directly related to the functionality being tested.

Listed below are the test cases for **EmpoweredAI** and Software System:

Field	Details
Test Case ID	TC001
Test Objective	Verify that a new user can register successfully.
Preconditions	1. The system is online and accessible. 2. The email being registered is not already in use.
Steps	1. Navigate to the registration page. 2. Enter the user details: Email Address, Password. 3. Click on the "Register" button.
Test Data	Email: hamzagondal921@gmail.com Password: SecurePass123
Expected Result	The system displays a message confirming registration success.
Post Condition	The user account is created and can be used for login in the future.
Actual Result	The user is registered.
Pass/Fail	Pass

Table 4.2: Test Case for User Registration

Field	Details
Test Case ID	TC002
Test Objective	Verify that a user cannot register with an already existing email address.
Preconditions	1. The system is online and accessible. 2. The email address being registered is already in use by another user.
Steps	1. Navigate to the registration page. 2. Enter the user details: Email Address, Password. 3. Click on the "Register" button.
Test Data	Email: hamzagondal921@gmail.com Password: SecurePass123
Expected Result	The system displays a message stating that the email is already in use.
Post Condition	No new account is created.
Actual Result	The system displays an error message indicating that the email is already taken.
Pass/Fail	Fail

Table 4.3: Test Case for Duplicate Email Registration

Field	Details
Test Case ID	TC003
Test Objective	Verify that a user can log in with valid credentials.
Preconditions	1. The system is online and accessible. 2. The user is registered in the system. 3. The user knows their credentials (username and password).
Steps	1. Navigate to the login page. 2. Enter a valid username and password. 3. Click on the "Login" button.
Test Data	Username: hamzagondal921@gmail.com Password: SecurePass123
Expected Result	The user is successfully logged in and redirected to the dashboard page.
Post Condition	The user session is active, and the dashboard is accessible.
Actual Result	The user is successfully navigated to the dashboard.
Pass/Fail	Pass

Table 4.4: Test Case: User Login

Field	Details
Test Case ID	TC004
Test Objective	Verify that a user cannot log in with an incorrect password.
Preconditions	1. The system is online and accessible. 2. The user is registered in the system. 3. The user knows their correct username but enters an incorrect password.
Steps	1. Navigate to the login page. 2. Enter a valid username and an incorrect password. 3. Click on the "Login" button.
Test Data	Username: hamzagondal921@gmail.com Password: WrongPassword123
Expected Result	The system displays an error message indicating that the password is incorrect or Username.
Post Condition	The user remains on the login page and is not logged in.
Actual Result	The system displays an error message stating incorrect password or Username.
Pass/Fail	Fail

Table 4.5: Test Case: Invalid Login Attempt

Field	Details
Test Case ID	TC005
Test Objective	Verify that the system correctly generates a scene description for the given image.
Preconditions	1. The system is online and accessible. 2. The image is uploaded or available for scene description processing.
Steps	1. Open the scene description feature. 2. Upload or display the image. 3. The system processes the image and generates a scene description.
Test Data	Image of an indoor swimming pool with the following details: - Pool in the center of the room. - Pebbles on the floor. - Metal rods visible. - Windows with curtains on the left and right sides. - A door in the background.
Expected Result	The system generates the following scene description: "The image shows an indoor swimming pool in the middle of a room with pebbles on the floor, metal rods, windows with curtains on the left and right sides, and a door in the background."
Actual Result	The system generates the scene description: "The image shows an indoor swimming pool in the middle of a room with pebbles on the floor, metal rods, windows with curtains on the left and right sides, and a door in the background."
Pass/Fail	Pass

Table 4.6: Test Case: Scene Description from Image

Bibliography

- [1] Flutter Community. flutter_tts plugin. https://pub.dev/packages/flutter_tts, 2024. Accessed: 2025-05-16.
- [2] Google Developers. ML kit image labeling and scene description. <https://developers.google.com/ml-kit/vision/image-labeling>, 2024. Accessed: 2025-05-16.
- [3] Google Developers. ML kit text recognition. https://pub.dev/packages/google_ml_kit, 2024. Accessed: 2025-05-16.
- [4] Flutter Team. Flutter documentation. <https://flutter.dev/docs>, 2024. Accessed: 2025-05-16.
- [5] Python Software Foundation. pyttsx3 text-to-speech conversion library. <https://pypi.org/project/pyttsx3/>, 2023. Accessed: 2025-05-16.
- [6] A Kolyshkin and S Nazarovs. Stability of slowly diverging flows in shallow water. *Mathematical Modeling and Analysis*, 2007.
- [7] Scikit learn Developers. Feature selection in python. https://scikit-learn.org/stable/modules/feature_selection.html, 2023. Accessed: 2025-05-16.
- [8] Ahmad M. tflite_flutter - tensorflow lite for flutter. https://github.com/am15h/tflite_flutter, 2023. Accessed: 2025-05-16.
- [9] MadMaze. pytesseract: Python wrapper for google's tesseract-ocr engine. <https://github.com/madmaze/pytesseract>, 2023. Accessed: 2025-05-16.
- [10] Flutter Plugins. speech_to_text plugin. https://pub.dev/packages/speech_to_text, 2024. Accessed: 2025-05-16.

- [11] Real Python. Color spaces and detection in opencv. <https://realpython.com/python-opencv-color-spaces/>, 2023. Accessed: 2025-05-16.
- [12] Roboflow. Train yolov8 on a custom dataset. <https://blog.roboflow.com/train-yolov8-custom-dataset/>, 2023. Accessed: 2025-05-16.
- [13] TensorFlow Team. Image classification with mobilenet on tflite. https://www.tensorflow.org/lite/models/image_classification/overview, 2024. Accessed: 2025-05-16.
- [14] Uberi. Speechrecognition python library. <https://pypi.org/project/SpeechRecognition/>, 2023. Accessed: 2025-05-16.
- [15] Ultralytics. Yolov8 documentation. <https://docs.ultralytics.com/>, 2023. Accessed: 2025-05-16.